

Système RAG MyGarage

DOCUMENTATION DE LA BASE DE CONNAISSANCES VECTORIELLE ET GUIDE D'ARCHITECTURE

Présentation du Projet

MyGarage is a Retrieval-Augmented Generation (RAG) system built for automotive diagnosis and maintenance. It transforms a structured car knowledge base into a searchable vector database, enabling an AI assistant to accurately answer questions about car symptoms, repairs, and maintenance schedules.

Fonctionnalités Principales

- **Segmentation (Chunking)** : Découpe 169 entrées de connaissances automobiles détaillées à partir du fichier source `car_final.txt`.
- **Vecteurs d'Embedding** : Transforme chaque segment de texte en un vecteur dense de 1024 dimensions en utilisant le modèle `mxbai-embed-large`.
- **Stockage en Base de Données** : Enregistre les données textuelles brutes ainsi que les vecteurs de haute dimension dans une base de données PostgreSQL 18 à l'aide de l'extension `pgvector`.
- **Indexation Performante** : Construit un index HNSW (Hierarchical Navigable Small World) pour assurer une exécution ultra-rapide des recherches approximatives des plus proches voisins (ANN).
- **Pipeline de Recherche Avancé** : Effectue une recherche par similarité vectorielle lors de la requête pour extraire les 10 meilleurs candidats (distance L2), puis applique un reclassement par `CrossEncoder` pour transmettre les 3 blocs de contexte optimaux au modèle LLM.

Structure des Fichiers

Fichier	Rôle	Fonctions Clés / Notes
<code>chunking_functions.py</code>	Logique principale de découpage	Contient les fonctions <code>embed()</code> , <code>create_chunk()</code> , <code>insert_chunk()</code> , et <code>create_hnsw_index()</code> .
<code>chunks_bd.py</code>	Modèle de données & configuration	Gère le mapping ORM SQLAlchemy, la déclaration des colonnes <code>pgvector</code> et la configuration du moteur de base de données.
<code>search_functions.py</code>	Pipeline de recherche et reclassement	Gère les fonctions <code>search_function()</code> , <code>rerank_results()</code> , et la logique principale de <code>retrieve_content()</code> .
<code>chunking.ipynb</code>	Notebook d'exécution	Orchestre l'intégralité du pipeline d'ingestion de données et d'indexation vectorielle de bout en bout.
<code>car_final.txt</code>	Source de la base de connaissances	Contient 169 entrées d'entretien et de diagnostic automobile, séparées explicitement par des jetons <code>[SEP]</code> .

Schéma de la Base de Données (Table : chunk)

Colonne	Type	Description
id	INTEGER (PK)	Clé primaire auto-incrémentée.
title	TEXT	Première ligne extraite de l'entrée de connaissances (ex: # Worn Brake Pads).
content	TEXT	Contenu textuel complet du segment de diagnostic.
embedding	VECTOR(1024)	Représentation vectorielle dense générée par le modèle mxbai-embed-large.

Pipeline Complet Étape par Étape

Étape	Fonction	Description	Technologie
1	extract_content()	Lit le texte brut structuré à partir du fichier source de la base de connaissances.	Python I/O
2	split(' [SEP] ')	Découpe le texte en entrées sémantiques distinctes selon les délimiteurs.	API String standard
3	embed()	Génère un vecteur dense à 1024 dimensions pour chaque segment de texte.	mxbai-embed-large
4	insert_chunk()	Enregistre les composants textuels et l'embedding vectoriel associé dans la base.	pgvector
5	create_hnsw_index()	Construit un index HNSW haute performance pour des requêtes ANN rapides.	pgvector HNSW
6	search_function()	Exécute une recherche de similarité par distance L2 pour extraire les 10 meilleurs résultats.	SQLAlchemy
7	rerank_results()	Reclasse la liste des candidats à l'aide d'un CrossEncoder pour isoler les 3 meilleures entrées.	BAAI/bge-reranker-base
8	retrieve_content()	Fournit le contexte final hautement pertinent et structuré à l'agent LLM.	Cœur du pipeline RAG

Stack Technologique

Couche	Technologie	Version / Notes d'implémentation
Moteur de Base de Données	PostgreSQL	Version 18, servant de magasin de données relationnelles principal.
Extension Vectorielle	pgvector	Permet le stockage et les opérations natives sur les colonnes VECTOR(1024).
Indexation Vectorielle	Index HNSW	Configured with L2 distance operations (vector_l2_ops).
Modèle d'Embedding	mxbai-embed-large	Exécution locale générant 1024 dimensions via Ollama.
Modèle de Reclassement	BAAI/bge-reranker-base	Implémentation CrossEncoder s'exécutant sur un périphérique accéléré par CUDA.
ORM / Pilote	SQLAlchemy	Associé au pilote de base de données psycopg (v3).
Environnement d'Exécution	Python 3.10	Orchestré via des notebooks Jupyter et des modules Python structurés.

Format de la Base de Connaissances (car_final.txt)

Chaque entrée de la base de données provient du fichier texte `car_final.txt`. Les segments sont séparés à l'aide du jeton [SEP]. La première ligne sert automatiquement de titre au segment, tandis que le bloc de texte suivant constitue le contenu complet du segment.

```
# Worn Brake Pads
Sound: Squealing, Screeching, Grinding
What Broke: Brake pads are too thin.
How to Recognize While Driving: Noise only when pressing brake pedal, Grinding means metal touching metal
Main Causes: Wear, Age
Is It Dangerous?: High Risk
Easy or Garage?: Garage recommended
Typical Fix: Replace brake pads (and rotors if scored)
```

Inspection de la Base de Données (Vérification des Données)

Voici les panneaux d'inspection des requêtes SQL extraits du tableau de bord client PostgreSQL 18, confirmant l'intégration des 169 segments et des embeddings correspondants.

pgAdmin IV - MyGarage/postgres@PostgreSQL 18

```
SELECT * FROM chunk;
```

Data Output | Messages | Notifications Lignes affichées : 1 à 169

id [PK]	title [text]	content [text]
1	# Worn Brake Pads	# Worn Brake Pads Sound: Squealing, Screeching, Grindin...
2	# Warped Brake Discs (Rotors)	# Warped Brake Discs (Rotors) Sound: Grinding, Pulsing ...
3	# Wheel Misalignment	# Wheel Misalignment Sound: Usually no loud sound, Tire...
4	# Tire Balancing Problem	# Tire Balancing Problem Sound: Humming, Vibration nois...
5	# Bad CV Joint (Cardan / Axle Joint)	# Bad CV Joint (Cardan / Axle Joint) Sound: Clicking, P...
6	# Bad Steering Rack	# Bad Steering Rack (Rack and Pinion) Sound: Clunking, ...
7	# Loose Tie Rod Ends	# Loose Tie Rod Ends Sound: Clunking, Knocking What Bro...

pgAdmin IV - MyGarage/postgres@PostgreSQL 18

```
SELECT "content", "embedding" FROM "chunk";
```

Data Output | Messages | Notifications Lignes affichées : 1 à 169

content [text]	embedding [vector]
# Worn Brake Pads Sound: Squealing, Screeching, Grin...	[-0.04682187, 0.03936265, 0.0124513, -0.01982...
# Warped Brake Discs (Rotors) Sound: Grinding, Pulsi...	[-0.03602896, 0.02629002, -0.0048123, 0.05129...
# Wheel Misalignment Sound: Usually no loud sound, T...	[-0.01236811, 0.04105354, 0.0891241, -0.03141...
# Tire Balancing Problem Sound: Humming, Vibration n...	[-0.04603844, 0.03114100, -0.0154812, 0.02294...
# Bad CV Joint (Cardan / Axle Joint) Sound: Clicking...	[-0.032168265, 0.0541450, 0.0014812, -0.06718...